If a clock *ticks at a uniform rate* (whether or not that rate matches the `period`) and *can't be adjusted*, the clock is said to be a *steady* clock. The `is_steady` static data member of the clock class is `true` if the clock is steady and `false` otherwise. Typically, `std::chrono::system_clock` will *not* be steady, because the clock can be adjusted, even if such adjustment is done automatically to take account of local clock drift. Such an adjustment may cause a call to `now()` to return a value earlier than that returned by a prior call to `now()`, which is in violation of the requirement for a uniform tick rate. Steady clocks are important for timeout calculations, as you'll see shortly, so the C++ Standard Library provides one in the form of `std::chrono::steady_clock`. The other clocks provided by the C++ Standard Library are `std::chrono::system_clock` (mentioned above), which represents the "real time" clock of the system and which provides functions for converting its time points to and from `time_t` values, and `std::chrono::high_resolution_clock`, which provides the smallest possible tick period (and thus the highest possible resolution) of all the library-supplied clocks. It may actually be a `typedef` to one of the other clocks. These clocks are defined in the `<chrono>` library header, along with the other time facilities.

We'll look at the representation of time points shortly, but first let's look at how durations are represented.

4.3.2. Durations

Durations are the simplest part of the time support; they're handled by the `std::chrono::duration<>` class template (all the C++ time-handling facilities used by the Thread Library are in the `std::chrono` namespace). The first template parameter is the type of the representation (such as `int`, `long`, or `double`), and the second is a fraction specifying how many seconds each unit of the duration represents. For example, a number of minutes stored in a `short` is `std::chrono::duration<short,std::ratio<60,1>>`, because there are 60 seconds in a minute. On the other hand, a count of milliseconds stored in a `double` is `std::chrono::duration<double,std::ratio<1,1000>>`, because each millisecond is 1/1000 of a second.

The Standard Library provides a set of predefined `typedef`s in the `std::chrono` namespace for various durations: `nanoseconds`, `microseconds`, `milliseconds`, `seconds`, `minutes`, and `hours`. They all use a sufficiently large integral type for the representation chosen such that you can represent a duration of over 500 *years* in the appropriate units if you so desire. There are also `typedef`s for all the SI ratios from `std::atto` (10^{-18}) to `std::exa` (10^{18}) (and beyond, if your platform has 128-bit integer types) for use when specifying custom durations such as `std::duration<double,std::centi>` for a count of 1/100 of a second represented in a `double`.

Conversion between durations is implicit where it does not require truncation of the value (so converting hours to seconds is OK, but converting seconds to hours is not). Explicit conversions can be done with `std::chrono::duration_cast<>`:

```
std::chrono::milliseconds ms(54802);
std::chrono::seconds s=
    std::chrono::duration_cast<std::chrono::seconds>(ms);
```